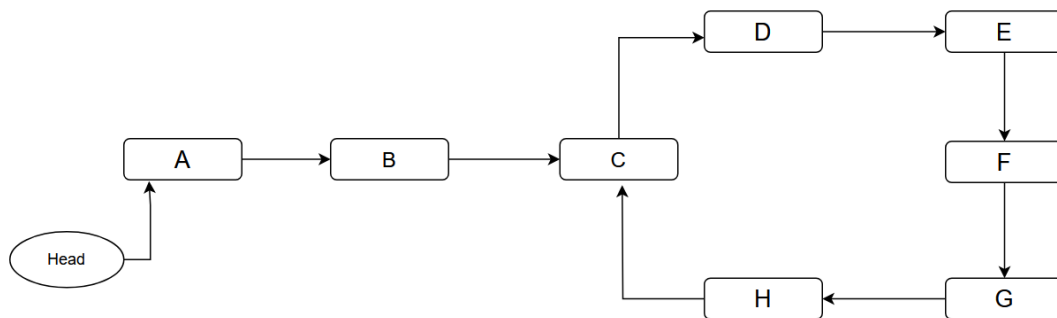


Floyd's Linked List Cycle Finding Algorithm

Given a linked list where the starting point of that linked list is denoted by **head**, and there may or may not be a cycle present. For instance:



Here we need to find out the point **C**, i.e the starting point of the cycle.

Proposed algorithm

The algorithm is called **Floyd's Cycle Algorithm or Tortoise And Hare algorithm**. In order to figure out the starting point of the cycle, we need to figure out if the cycle even exists or not. So, it involved two steps: 1. Figure out the presence of the cycle. 2. Find out the starting point of the cycle.

Step 1: Presence of the cycle

1. Take two pointers `$slow$` and `$fast$`.
2. Both of them will point to head of the linked list initially.
3. `$slow$` will move one step at a time.
4. `$fast$` will move two steps at a time. (twice as speed as `$slow$` pointer).
5. Check if at any point they point to the same node before any one(or both) reach null.
6. If they point to any same node at any point of their journey, it would indicate that the cycle indeed exists in the linked list.
7. If we get null, it would indicate that the linked list has no cycle.